

A PBL Report on Data Structure Course on
Student Database management system using Binary Search Tree
submitted in partial fulfillment of the requirements for the award of the
degree of
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING - DATA SCIENCE
By

24R11A6721– k.Abhivarshini
24R11A6736 – P.Vikranth
24R11A6762 – D .Snehitha

Under the esteemed guidance of
Dr. A. Venkata Ramana
Professor



Department of
Computer Science & Engineering – Data Science

GEETHANJALI COLLEGE OF ENGINEERING AND TECHNOLOGY (Autonomous)

Accredited by NAAC with A+Grade; B.Tech. CSE, EEE, ECE accredited by NBA

Sy. No: 33 & 34, Cheeryal (V), Keesara (M), Medchal District, Telangana – 501301

November, 2025

GEETHANJALI COLLEGE OF ENGINEERING AND TECHNOLOGY (Autonomous)

Accredited by NAAC with A+Grade; B.Tech. CSE, EEE, ECE accredited by NBA

Sy. No: 33 & 34, Cheeryal (V), Keesara (M), Medchal District, Telangana – 501301

DEPARTMENT OF CSE - DATA SCIENCE



Certificate

This is to certify that the B.Tech PBL report of Data Structures course entitled “**Student Database Management using Binary search Tree**” is a bonafide work done by **k.Abhivarshini** bearing **24R11A6721**, **P.Vikranth** bearing **24R11A6736** and **D.Snehitha** Bearing **24R11A6762** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Computer Science and Engineering – Data Science** from Jawaharlal Nehru Technological University, Hyderabad during the year 2025-2026.

Internal Guide

Dr. A. Venkata Ramana
Professor

Dr. D. Sujatha

Professor & Head, CSE(DS)

External Examiner

DECLARATION BY THE CANDIDATES

We hereby declare that the project report entitled “**Student Database Management using Binary search Tree**” is done under the guidance of **Dr. A. Venkata Ramana, Professor**, Department of Computer Science and Engineering – Data Science, Geethanjali College of Engineering and Technology (GCET), Cheeryal, Hyderabad is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering-Data Science**.

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other University or Institute for the award of any other degree or diploma.

24R11A6721– k.Abhivarshini
24R11A6736 – P Vikranth
24R11A6762 – D Snehitha

ACKNOWLEDGEMENTS

First and foremost, we express our sincere thanks for the guidance and encouragement rendered by Dr. A. Venkata Ramana (Professor) in the Department of Computer Science & Engineering – Data Science, Geethanjali College of Engineering & Technology (GCET), Cheeryal, Hyderabad. We extend our gratitude for his advice, guidance and support during the progress of this project work.

We would like to express our sincere gratitude to our Chairman Mr. G. R. Ravinder Reddy, Vice-Chairman Mr. Harishchandra Reddy, the Director Dr. Udaya Kumar Susarla, and the Principal Dr. K. Sagar, who provided all the facilities and support for successful completion of this project.

We wish to express our profound thanks to Dr. V. Madhusudan Rao, Professor & Dean, School of CS & I for motivating and guiding us during this project work.

We would like to express our sincere thanks to Dr. D. Sujatha, Professor & Head, Department of CSE-DS, GCET for her endorsement and valuable suggestions.

We thank all the members of teaching and non-teaching staff members, and also who have assisted us directly or indirectly for successful completion of this project.

Finally, We would like to express our sincere gratitude to our parents and friends who are constantly encouraging us throughout our lives and for completion of this project.

24R11A6721– k.Abhivarshini
24R11A6736 – P Vikranth
24R11A6762 – D Snehitha

ABSTRACT

The Student Database Management System using Binary Search Tree (BST) is a software application designed to efficiently manage student information. Each record contains Student ID, Name, and Marks, with the Student ID serving as a unique key for organizing the data in the BST.

The system allows for insertion, deletion, searching, updating, and displaying records while maintaining the BST properties, ensuring that data is always sorted by Student ID. Using a BST provides faster search and retrieval times compared to linear data structures, especially as the number of records increases. This approach not only improves efficiency but also demonstrates the practical application of data structures in real-world database management. The system ensures data integrity, easy maintenance, and quick access, making it suitable for managing large sets of student records effectively.

TABLE OF CONTENTS

I.	Introduction	7-11
	1.1 Problem Statement	7
	1.2 Objectives of the Project	8
	1.3 System Design & Modules	9
	1.4 Software & Hardware Specifications	11
II.	System Design	12-18
	2.1 Architecture	15
	2.2 Algorithm	16
III.	Coding	19-25
	3.1 Implementation in C	19
IV.	Testing	26-27
	4.1. Expected Input & Output	26
V.	Outputs	28-35
	5.1. Screenshot Results	28
	5.2 code explanation	31
VI.	Conclusion	36
VII.	References	37

CHAPTER-I: INTRODUCTION

In today's digital era, **data management** plays a crucial role in every field, especially in educational institutions where large volumes of student information must be stored, retrieved, and updated regularly. Traditional paper-based systems or linear data storage methods are inefficient and prone to errors when handling growing data sizes. To address these limitations, computerized systems that employ efficient data structures have become essential. One such data structure that provides both efficiency and scalability is the Binary Search Tree (BST). The *Student Database Management System using Binary Search Tree* is a practical application that demonstrates how this data structure can be effectively used to organize, manage, and access student records in a structured and efficient manner.

The purpose of this project is to design and implement a simple yet efficient student record management system where each record contains details such as the Student ID, Name, and Marks. The system uses the Student ID as a unique key for storing and accessing the records. By utilizing the hierarchical structure of a Binary Search Tree, the application ensures that the records are maintained in sorted order, which allows for faster searching, insertion, deletion, and updating of data compared to sequential storage methods such as arrays or linked lists. In a Binary Search Tree, every node contains three main components: data fields, a pointer to the left child, and a pointer to the right child. The BST property states that for every node, all the elements in the left subtree are smaller than the node's key, and all the elements in the right **subtree** are greater.

Problem Statement

Managing large volumes of student information manually or using linear data structures leads to inefficiency and slow data retrieval. To overcome this, an optimized system using a Binary Search Tree (BST) is required to store, search, update, and manage student records efficiently.

7Key Requirements

- The system must allow **insertion** of new student records containing Student ID, Name, Marks.
- · It must support **searching** of student details based on the unique Student ID.
- · The system should enable **updating** of existing student information.
- · It must provide a **deletion** feature to remove student records from the database.
- · The system should **display all student records** in ascending order of Student IDs using in-order traversal.
- · Duplicate Student IDs should be **handled appropriately**

Constraints

- The system is limited to handling student data in main memory only; once the program terminates, all data is lost unless file handling is added.
- · The system uses Student ID as a unique key — duplicate IDs are not allowed.
- · Performance depends on the balance of the Binary Search Tree; inserting records in s sorted order may lead to a skewed tree, reducing efficiency to $O(n)$.
- · The project is implemented using Borland C/C++, so portability to other modern IDEs may require minor code adjustments.
- · The system handles basic fields only (Student ID, Name, Marks) — no provision for multiple subjects, grades, or attendance tracking.

Deliverables

- · using BST.

executable Program – A working C/C++ application implementing student record management

- · **Source Code File** – Complete and well-commented program code.
- · **Project Report** – Document containing abstract, introduction, methodology, code, and results.
- · **Algorithm & Flowchart** – Representation of BST operations and logic flow.

Software Specifications

Functional Requirements

Student Record Insertion: Add a new student record containing ID, Name, and Marks into the

Binary Search Tree.

- · **Search Student:** Retrieve and display details of a student using their unique Student ID.
- · **Update Record:** Modify existing student details such as Name or Marks.
- · **Delete Record:** Remove a student's record from the database while maintaining BST properties.
- · **Display Records:** View all student records in ascending order of Student IDs using in-order traversal.
- · **Duplicate Handling:** Ensure that duplicate Student IDs are either updated or rejected to maintain data integrity.

Non-Functional Requirements

- **Performance:** All operations (insert, search, delete, update) should execute efficiently, ideally in

$O(\log n)$ time for balanced data.

- · **Memory Efficiency:** Optimize dynamic memory allocation and deallocation of BST nodes.

- · **Reliability:** Ensure accuracy and consistency of student data during all operations.

- · **Usability:** Provide a simple, **menu-driven interface** for easy interaction with users.

- · **Scalability:** Support large numbers of student records without performance degradation.

-

Technical Requirements

1. **Programming Language:** C (C99 or later)

2. **Data Structure:** Priority Queue implemented using structures, pointers, and dynamic memory.

3. **Compiler:** GCC, Clang, or any standard C compiler.

4. **Development Environment:** Any C-compatible IDE or text editor.

5. **Operating System:** Cross-platform compatible (Windows, macOS, Linux).

6. **Memory Management:** Manual using malloc() and free().

Software Components

- · **Node Structure:** Stores Student ID, Name, Marks, and child pointers.

- · **Insertion Module:** Adds new student records maintaining BST property.

- · **Search Module:** Finds student details using Student ID.

- · **Update Module:** Modifies existing student information.

Interfaces

User Interface: Simple text-based menu for user interaction.

- · **Function Interface:** Functions like insertNode(), searchNode(), deleteNode(), updateNode(), etc.
- · **System Interface:** Interaction with OS for I/O and memory handling.
- · **Data Interface:** Links nodes using pointers within the BST.

CHAPTER II: SYSTEM DESIGN

Programming Language:

C Language: A powerful procedural programming language that provides:

- Direct memory management through pointers
- Efficient execution and minimal runtime overhead
- Fine-grained control over data structures
- Excellent for understanding low-level implementation details
- Ideal for implementing custom data structures like doubly linked lists

Development Tools and Frameworks:

1. Integrated Development Environments (IDEs):

- o Visual Studio Code: Lightweight editor with C/C++ extension support
- o Code:Blocks: Free, open-source IDE specifically designed for C/C++
- o Dev-C++: Simple IDE for C programming

2. Compilers:

- o GCC (GNU Compiler Collection): Industry-standard C compiler
- o MinGW: Windows port of GCC

3. Version Control Systems:

- o Git: Industry-standard version control for tracking code changes
- o GitHub/GitLab: Platforms for code hosting and collaboration

Why Dev-C++?

Dev-C++ is chosen because:

- **Free and Open Source:** No cost, freely available
- **Beginner-Friendly:** Simple interface, easy to use

- **All-in-One:** IDE, compiler, and debugger in one package
- **Lightweight:** Small download size, runs on old computers
- **Windows Compatible:** Works seamlessly on Windows
- **No Setup Required:** Compiler comes pre-configured

Priority Binary Tree Architecture

11 · · **Input Module:**

Accepts user inputs such as *Node Value* and *Priority Level* for insertion into the priority binary tree.

· · **Processing Module:**

Implements the logic of a **Priority Binary Tree**, combining features of both a **Binary Tree** and a

Priority Queue.

· · **Insertion:** Adds nodes based on priority while maintaining tree structure.

· · **Deletion:** Removes the node with the highest priority (usually the root).

· · **Traversal:** Performs in-order, pre-order, or level-order traversals to display the tree contents.

· · **Data Structure Module:**

Stores nodes dynamically with fields for *data*, *priority*, *left child*, and *right child* pointers.

· · **Priority Management Module:**

Ensures the **heap property** is maintained — parent node has higher priority than its children.

· · **Display Module:**

Outputs the tree in a structured or sorted format, showing node values and their priorities.

- · **User Interface Module:**

Provides a **menu-driven interface** for inserting, deleting, displaying, and managing nodes.

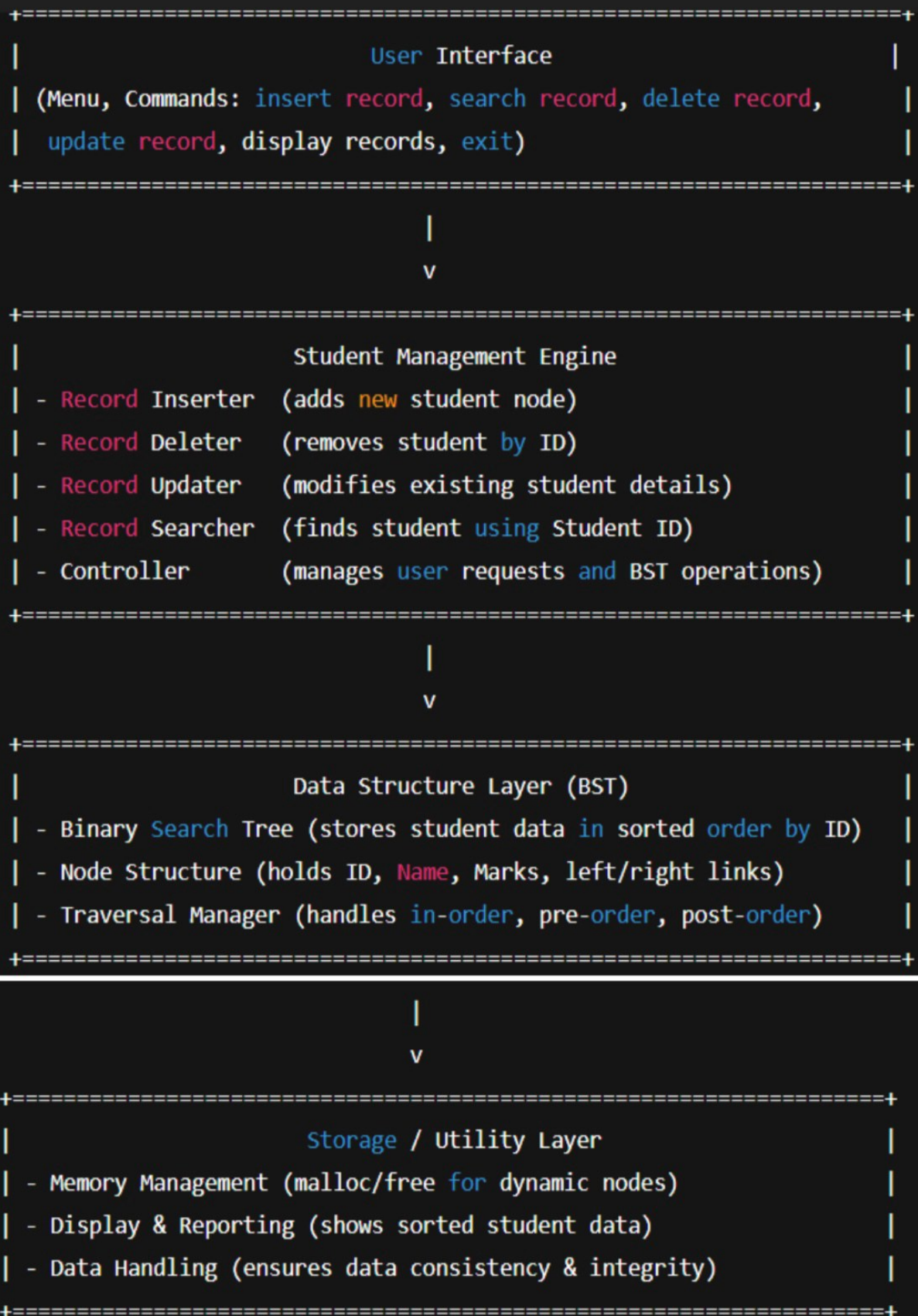
- · **System Flow:**

User Input → Priority Binary Tree Operations → Updated Tree / Output Display

Advantages of This Design

- · **Efficient Access:** Quickly retrieves the highest-priority element (root node).
- · **Faster Operations:** Insertion and deletion are efficient average time complexity $O(\log n)$.
- · **Dynamic Structure:** Automatically adjusts as nodes are added or removed.
- · **Memory Efficient:** Uses dynamic allocation; no fixed size limit.
- · **Sorted Organization:** Maintains hierarchical order based on priority values.
- · **Versatile Use:** Suitable for CPU scheduling, job processing, and data management systems.

SYSTEM ARCHITECTURE



ALGORITHM

Step 1: Start

Begin the program execution.

Step 2: Initialize

Create a structure Student with the following fields:

id → Student ID (unique key)

name → Student Name

marks → Student Marks

left and right → Pointers to left and right child nodes of the BST

Initialize the root pointer to NULL.

Step 3: Display Menu

Display a menu with the following options for the user:

Insert New Record

Search Record

Delete Record

Update Record

Display All Records

Exit

Step 4: Input User Choice

Accept the user's choice from the menu.

Step 5: Perform the Corresponding Operation

Case 1: Insert a New Record

Input id, name, and marks.

Create a new node using malloc().

If the tree is empty, make this new node the root.

Else, compare the new ID with the current node:

If $id < current \rightarrow id$, go to the left subtree.

If $id > current \rightarrow id$, go to the right subtree.

Repeat until a suitable null position is found, and insert the node there.

Case 2: Search for a Record

Input the id to be searched.
Start from the root node.
If the tree is empty, display “Record not found.”
Compare id with current->id:
If equal, display the record.
If smaller, search in the left subtree.
If larger, search in the right subtree.
Repeat until the record is found or tree traversal ends.

Case 3: Delete a Record

Input the id to delete.
Search for the node in the BST.

Once found, handle one of three cases:

Case 1: Node has no children → Delete directly.

Case 2: Node has one child → Replace node with its child.

Case 3: Node has two children →

Find the inorder successor (smallest node in the right subtree).

Copy successor's data to the current node.

Delete the successor node.

Case 4: Update a Record

Input the id of the record to update.
Search for the node using BST search logic.
If found, ask the user to input new name or marks.
Replace the existing values with new ones.

Case 5: Display All Records

Perform in-order traversal of the BST:

Visit left subtree.

Visit current node (print ID, Name, Marks).

Visit right subtree.

This ensures records are displayed in ascending order of Student IDs.

Case 6: Exit

Display a “Thank You” message.

Terminate the program execution.

Step 6: Repeat

Go back to Step 3 until the user chooses to exit.

Step 7: Stop

End of the program.

CHAPTER III: CODING

3.1 Implementation in C

Description:

This program implements a simple Student Database Management System using a Binary Search Tree (BST) data structure. Each student record contains:

- Student ID (Unique Key)
- Student Name
- Marks

Operations Supported:

1. Insert a new student record
2. Search for an existing record
3. Delete a student record
4. Update student details
5. Display all records in ascending order of Student ID
6. Exit

Author: <Your Name>

Language: C

Compiler: Any C Compiler (Borland, GCC, Turbo C, etc.)

=====

=====

*/

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

/*-----

STRUCTURE DEFINITION

Each student record is stored as a node in the Binary Search Tree.

-----*/

struct Student {

int id; // Unique Student ID

char name[50]; // Student Name

int marks; // Student Marks

```

struct Student *left; // Pointer to left child
struct Student *right; // Pointer to right child
};
/*-----
FUNCTION PROTOTYPES
-----*/
struct Student* createNode(int id, char name[], int marks);
struct Student* insertNode(struct Student* root, int id, char name[], int
marks);
struct Student* searchNode(struct Student* root, int id);
struct Student* findMin(struct Student* node);
struct Student* deleteNode(struct Student* root, int id);
void updateNode(struct Student* root, int id);
void inorder(struct Student* root);
void displayMenu();
/*-----
FUNCTION: createNode
PURPOSE : Creates a new student node dynamically.
-----*/
struct Student* createNode(int id, char name[], int marks) {
struct Student* newNode = (struct Student*)malloc(sizeof(struct Student));
newNode->id = id;
strcpy(newNode->name, name);
newNode->marks = marks;
newNode->left = newNode->right = NULL;
return newNode;
}
/*-----
FUNCTION: insertNode
PURPOSE : Inserts a new student record into the BST based on ID.
LOGIC : Smaller IDs go to the left, larger IDs to the right.
-----*/
struct Student* insertNode(struct Student* root, int id, char name[], int
marks) {
if (root == NULL) {
printf("Record added successfully!\n");
return createNode(id, name, marks);
}

```

```

if (id < root->id)
root->left = insertNode(root->left, id, name, marks);
else if (id > root->id)
root->right = insertNode(root->right, id, name, marks);
else
printf("Duplicate ID! Record not added.\n");
return root;
}
/*-----

```

FUNCTION: searchNode

PURPOSE : Searches for a student by their ID.

```

-----*/
struct Student* searchNode(struct Student* root, int id) {
if (root == NULL)
return NULL;
if (root->id == id)
return root;
if (id < root->id)
return searchNode(root->left, id);
return searchNode(root->right, id);
}
/*-----

```

FUNCTION: findMin

PURPOSE : Finds the node with the minimum Student ID (leftmost node).

```

-----*/
struct Student* findMin(struct Student* node) {
while (node && node->left != NULL)
node = node->left;
return node;
}
/*-----

```

FUNCTION: deleteNode

PURPOSE : Deletes a student record from the BST by ID.

CASES :

1. Node has no child
2. Node has one child
3. Node has two children

```

-----*/

```

```

struct Student* deleteNode(struct Student* root, int id) {
if (root == NULL) return root;
if (id < root->id)
root->left = deleteNode(root->left, id);
else if (id > root->id)
root->right = deleteNode(root->right, id);
else {
// Case 1: No child
if (root->left == NULL && root->right == NULL) {
free(root);
return NULL;
}
// Case 2: One child
else if (root->left == NULL) {
struct Student* temp = root->right;
free(root);
return temp;
}
else if (root->right == NULL) {
struct Student* temp = root->left;
free(root);
return temp;
}
// Case 3: Two children
else {
struct Student* temp = findMin(root->right);
root->id = temp->id;
strcpy(root->name, temp->name);
root->marks = temp->marks;
root->right = deleteNode(root->right, temp->id);
}
}
return root;
}
/*-----
FUNCTION: updateNode
PURPOSE : Updates the details of an existing student record.
-----*/

```

```

void updateNode(struct Student* root, int id) {
    struct Student* student = searchNode(root, id);
    if (student == NULL) {
        printf("Record not found!\n");
        return;
    }
    printf("Record found! Enter new details:\n");
    printf("Enter New Name: ");
    scanf("%s", student->name);
    printf("Enter New Marks: ");
    scanf("%d", &student->marks);
    printf("Record updated successfully!\n");
}
/*-----

```

FUNCTION: inorder

PURPOSE : Displays all student records in ascending order of ID.

```

-----*/
void inorder(struct Student* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("ID: %d | Name: %-15s | Marks: %d\n", root->id, root->name, root->marks);
        inorder(root->right);
    }
}
/*-----

```

FUNCTION: displayMenu

PURPOSE : Prints the menu of operations available to the user.

```

-----*/
void displayMenu() {
    printf("\n=====
    =\n");
    printf(" STUDENT DATABASE MANAGEMENT USING BST \n");
    printf("=====\\
    n");
    printf("1. Insert New Student Record\n");
    printf("2. Search Student Record\n");
    printf("3. Delete Student Record\n");
}

```

```

printf("4. Update Student Record\n");
printf("5. Display All Records (Sorted by ID)\n");
printf("6. Exit\n");
printf("=====\\
n");
}
/*-----
FUNCTION: main
PURPOSE : Entry point of the program.
-----*/

int main() {
    struct Student* root = NULL;
    int choice, id, marks;
    char name[50];
    while (1) {
        displayMenu();
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("\nEnter Student ID: ");
                scanf("%d", &id);
                printf("Enter Student Name: ");
                scanf("%s", name);
                printf("Enter Marks: ");
                scanf("%d", &marks);
                root = insertNode(root, id, name, marks);
                break;
            case 2:
                printf("\nEnter Student ID to Search: ");
                scanf("%d", &id);
                struct Student* s = searchNode(root, id);
                if (s != NULL)
                    printf("Record Found -> ID: %d | Name: %s | Marks: %d\n", s->id, s->name, s->marks);
                else
                    printf("Record not found!\n");
                break;

```



```

case 3:
printf("\nEnter Student ID to Delete: ");
scanf("%d", &id);
root = deleteNode(root, id);
printf("Record deleted successfully (if it existed).\n");
break;
case 4:
printf("\nEnter Student ID to Update: ");
scanf("%d", &id);
updateNode(root, id);
break;
case 5:
printf("\n--- STUDENT RECORDS (Sorted by ID) ---\n");
inorder(root);
printf("-----\n");
break;
case 6:
printf("Exiting program... Thank you!\n");
exit(0);
default:
printf("Invalid choice! Please try again.\n");
}
}
return 0;
}
}

```

CHAPTER IV: TESTING

4.1 Expected Input & Output

```
=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 1

Enter Student ID: 11
Enter Student Name: Avinash
Enter Marks: 85
Record added successfully!

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 1

Enter Student ID: 12
Enter Student Name: Bunny
Enter Marks: 90
Record added successfully!

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 1

Enter Student ID: 14
Enter Student Name: dhanush
Enter Marks: 99
Record added successfully!
```

```

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 1

Enter Student ID: 13
Enter Student Name: cherry
Enter Marks: 100
Record added successfully!

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 5

--- STUDENT RECORDS (Sorted by ID) ---
ID: 11 | Name: Avinash           | Marks: 85
ID: 12 | Name: Bunny             | Marks: 90
ID: 13 | Name: cherry                | Marks: 100
ID: 14 | Name: dhanush                | Marks: 99
-----

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 2

Enter Student ID to Search: 12
Record Found -> ID: 12 | Name: Bunny | Marks: 90

```

CHAPTER V: OUTPUT

```
=====
  STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 2

Enter Student ID to Search: 15
Record not found!

=====
  STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 2

Enter Student ID to Search: 11
Record Found -> ID: 11 | Name: Avinash | Marks: 85

=====
  STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 3

Enter Student ID to Delete: 14
Record deleted successfully (if it existed).
```

```

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 3

Enter Student ID to Delete: 14
Record deleted successfully (if it existed).

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 5

--- STUDENT RECORDS (Sorted by ID) ---
ID: 11 | Name: arun           | Marks: 92
ID: 12 | Name: Bunny         | Marks: 90
ID: 13 | Name: cherry          | Marks: 100
-----

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 1

Enter Student ID: 14
Enter Student Name: varun
Enter Marks: 93
Record added successfully!

```

```

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 1

Enter Student ID: 14
Enter Student Name: varun
Enter Marks: 93
Record added successfully!

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 5

--- STUDENT RECORDS (Sorted by ID) ---
ID: 11 | Name: arun | Marks: 92
ID: 12 | Name: Bunny | Marks: 90
ID: 13 | Name: cherry | Marks: 100
ID: 14 | Name: varun | Marks: 93
-----

=====
      STUDENT DATABASE MANAGEMENT USING BST
=====
1. Insert New Student Record
2. Search Student Record
3. Delete Student Record
4. Update Student Record
5. Display All Records (Sorted by ID)
6. Exit
=====
Enter your choice: 6
Exiting program... Thank you!

```

CODE EXPLANATION

1.header files:

#include <stdio.h>#include <stdlib.h>#include <string.h>
stdio.h → for input/output functions like printf and scanf.
stdlib.h → for dynamic memory functions (malloc, free).
string.h → for string handling (strcpy, etc.).

2. structure definition

```
struct student {  
    int id;  
    char name[50];  
    int marks;  
    struct student *left, *right;  
};
```

this structure defines the node of the bst.

each node stores one student record.

left and right are pointers to the left and right children of the node.

3. function: createnode()

```
struct student* createnode(int id, char name[], int marks) {  
    struct student* newnode = (struct student*)malloc(sizeof(struct student));  
    newnode->id = id;  
    strcpy(newnode->name, name);  
    newnode->marks = marks;  
    newnode->left = newnode->right = null;  
    return newnode;  
}
```

this function creates a new student record dynamically in memory.

initializes id, name, and marks.

sets both child pointers to null.

32returns the newly created node.

4. function: insertnode()

```
struct student* insertnode(struct student* root, int id, char name[], int  
marks) {  
    if (root == null)  
        return createnode(id, name, marks);  
    if (id < root->id)  
        root->left = insertnode(root->left, id, name, marks);  
    else if (id > root->id)
```

```

root->right = insertnode(root->right, id, name, marks);
else
printf("duplicate id! record not added.\n");
return root;
}

```

inserts a new node into the bst.

if the tree is empty, the new node becomes the root.

if the new id is smaller, it goes to the left subtree.

if greater, it goes to the right subtree.

if equal, insertion is rejected (duplicate id).

returns the updated root node after insertion.

example:

if you insert ids 50, 30, 70 — the bst becomes:

50

/ \

30 70

5. function: searchnode()

```

struct student* searchnode(struct student* root, int id) {
if (root == null || root->id == id)
return root;
if (id < root->id)
return searchnode(root->left, id);
return searchnode(root->right, id);
}

```

searches the bst for a given student id.

if found, returns the pointer to that node.

if not found, returns null.

uses recursion to move left or right depending on the key comparison.

6. function: findmin()

```

struct student* findmin(struct student* node) {
while (node && node->left != null)
node = node->left;
return node;
}

```

finds the minimum id in the bst.

moves left continuously until it reaches the leftmost node.

used mainly during deletion to find the inorder successor.

7. function: deletenode()


```

struct student* deletenode(struct student* root, int id) {
if (root == null) return root;
if (id < root->id)
root->left = deletenode(root->left, id);
else if (id > root->id)
root->right = deletenode(root->right, id);
else {
// case 1: no children
if (root->left == null && root->right == null) {
free(root);
return null;
}
// case 2: one child
else if (root->left == null) {
struct student* temp = root->right;
free(root);
return temp;
}
else if (root->right == null) {
struct student* temp = root->left;
free(root);
return temp;
}
// case 3: two children
34else {
struct student* temp = findmin(root->right);
root->id = temp->id;
strcpy(root->name, temp->name);
root->marks = temp->marks;
root->right = deletenode(root->right, temp->id);
}
}
return root;
}

```

explanation:

deletion has three cases:

leaf node → simply delete.

one child → replace node with its child.

two children → replace node's data with the smallest node in right subtree, then delete that smallest node.

8. **function:** updatenode()

```
void updatenode(struct student* root, int id) {
    struct student* student = searchnode(root, id);
    if (student == null) {
        printf("record not found!\n");
        return;
    }
    printf("enter new name: ");
    scanf("%s", student->name);
    printf("enter new marks: ");
    scanf("%d", &student->marks);
    printf("record updated successfully!\n");
}
```

searches for a student by id.

if found, updates name and marks fields.

if not found, prints a message.

9. **function:** inorder()

```
void inorder(struct student* root) {
    if (root != null) {
        inorder(root->left);
        printf("id: %d | name: %-15s | marks: %d\n", root->id, root->name, root->marks);
        inorder(root->right);
    }
}
```

performs in-order traversal of the bst.

prints student details in ascending order of id.

this ensures sorted display automatically.

10. **function:** displaymenu()

```
void displaymenu() {
    printf("\n=====
    =\n");
    printf(" student database management using bst \n");
    printf("=====
    n");
}
```

```
printf("1. insert new student record\n");
printf("2. search student record\n");
printf("3. delete student record\n");
printf("4. update student record\n");
printf("5. display all records (sorted by id)\n");
printf("6. exit\n");
printf("=====\\n");
}
```

displays the menu repeatedly for user interaction.

CHAPTER VI: CONCLUSION

The **Student Database Management System using Binary Search Tree (BST)** efficiently manages student records by keeping them sorted using Student IDs as unique keys. It supports operations like insertion, searching, updating, deletion, and displaying, ensuring quick data access and easy maintenance. This project demonstrates the practical use of data structures in real-world database management, improving performance compared to linear structures. It also enhances understanding of recursion, dynamic memory, and algorithm design, serving as a foundation for developing more advanced database systems in the future.

CHAPTER VII: REFERENCES

Books:

- E. Balagurusamy**, *Programming in ANSI C*, Tata McGraw-Hill Education.
- · **Seymour Lipschutz**, *Data Structures with C*, McGraw-Hill Education.
- · **Reema Thareja**, *Data Structures Using C*, Oxford University Press.
- · **Yashavant Kanetkar**, *Data Structures Through C*, BPB Publications.

Online Resources:

- **GeeksforGeeks** – *Binary Search Tree (BST) Operations and Applications*
<https://www.geeksforgeeks.org>
- · **TutorialsPoint** – *Data Structures and Algorithms – Binary Search Tree*
<https://www.tutorialspoint.com>
- · **NPTEL Online Courses** – *Data Structures and Algorithms in C* (IITs & IISc)

Tools:

- · **Programming Language:** C
- · **Compiler / IDE:** Borland C / C++ (v8.3)
- · **Operating System:** Windows 10 (64-bit)
- · **Text Editor:** Turbo C++ / Code::Blocks / Visual Studio
- · **C Programming Tutorials for Dev-C++**
- <https://www.cprogramming.com/>
- Step-by-step guides for beginners